The caveat is, it's hard to know or even estimate how many nodes a trie will have based on the number of keys it stores; if, however, we know it contains `n` keys whose maximum length is `M`, then the worst-case (loose) bound for a non-empty string is `O(m + n*(M-m))`, corresponding to a degenerate trie where all words share exactly the prefix searched, and no further character.

In the example shown in figure 6.10, we search for all keys matching prefix "`ant`", so `n=6`, `m=3` and `M=8` (the length of the longest key, "`antidote`").

If, instead, we search all keys starting whose prefixes include the empty string, this will return all keys in the tree, and the running time will be `O(n*M)`, which would also be the worst-case upper bound for the method.

6.2.7 When Should We Use Tries?

Now that we have described all the main methods on tries, it feels like taking a moment to recap would be a good idea; table 6.2 shows the performance of tries on these methods, compared to the equivalent methods for balanced BSTs.

Table 6.2 helps answering the question we have put on hold in section 6.2.1: we had seen when a trie would require less memory than a BST, now we also know that it would almost always be faster.

Remember that, while in general we express the running time for BSTs just in terms of `n`, the number of entries stored, in this case we can't assume that the cost to compare two